

Assignment 2: Lexer and Parser for RustY

IC526 Advanced Compilers

Fall 2026

Objective

Implement a lexer and a parser for RustY, the small Rust-like language used in this course. Use `ocamllex` to recognize tokens and `Menhir` to construct the abstract syntax tree (AST) defined in `lib/ast.ml`. This assignment practices regular expressions, context-free grammars, semantic actions, operator precedence, and associativity.

1. Languages

Lexical rules

- Identifiers start with an ASCII letter or `_`, followed by zero or more ASCII letters, digits, or underscores. Names are case-sensitive. `_` is an ordinary identifier.
- Integer literals are nonempty sequences of decimal digits. Leading zeros are allowed: `012` denotes decimal 12. Their values are carried by `INT` as OCaml `int` values. Assume that tested literals fit in OCaml `int`. A minus sign is a separate token: `-12` produces `MINUS` followed by `INT 12`.
- Skip spaces, tabs, carriage returns, and line feeds. Call `Lexing.new_line lexbuf` when consuming a line feed. Skip `//` comments through the end of the line or end of input; still update the line position for the terminating line feed.

Tokens

The token names below are declared in `lib/parser.mly`. The spellings in each row correspond to the token names in the same order.

Source spelling	Token
<code>fn let mut return</code>	<code>FN LET MUT RETURN</code>
<code>i32 usize bool</code>	<code>I32 USIZE BOOL</code>
<code>if else while</code>	<code>IF ELSE WHILE</code>
<code>println!</code> and the exact string <code>"{"</code>	<code>PRINTLN</code> and <code>FMTSTR</code>
<code>true false</code>	<code>TRUE FALSE</code>
<code>() { } []</code>	<code>LPAREN RPAREN LBRACE RBRACE LBRACKET RBRACKET</code>
<code>, : ; = - ></code>	<code>COMMA COLON SEMICOLON EQ ARROW</code>
<code>* & + - / %</code>	<code>STAR AMP PLUS MINUS SLASH PERCENT</code>
<code>< > <= >= == !=</code>	<code>LT GT LE GE EQEQ NE</code>
<code>&& !</code>	<code>AMPAMP OROR NOT</code>
Decimal digits; an identifier	<code>INT n</code> ; <code>IDENT name</code>
End of input	<code>EOF</code>

`println!` is one contiguous token; `println !` is different. The same `STAR` token is used for multiplication and dereference, and the same `AMP` token for reference types and reference arguments. Their roles are determined by the parser.

There are no general string literals, floating-point literals, or block comments. Report an unsupported character using the supplied `LexError` exception and its starting `Lexing.position`; the exact error message is not prescribed. A comment that reaches end of input without a final newline is valid.

Program structure and types

A program is a list of zero or more function definitions followed by end of input. A function has a name, a comma-separated parameter list, an optional return type, and a body. Each parameter has an explicit type. For example:

```
fn f(x: i32, a: &[i32; 3], p: &mut i32) -> i32 {
    return x;
}
```

Value types are `i32`, `usize`, `bool`, and `[T; n]`, where `T` is a value type and `n` is an integer literal. Arrays may be nested. Parameter types are `T`, `&T`, and `&mut T`. Local type annotations and explicit return types use value types. Omitting the return annotation gives `T_unit` in the AST; explicit `()` types and unit literals are not supported.

Expressions and arguments

Here, `x` and `f` denote identifiers, and `e`, `a`, and `i` denote expressions.

Form	Supported syntax
Literals and variables	Integer literals, <code>true</code> , <code>false</code> , <code>x</code>
Dereference and arrays	<code>*x</code> , <code>[]</code> , <code>[e1, e2, ...]</code> , <code>a[i]</code>
Unary and binary operations	<code>-e</code> , <code>!e</code> , and the binary operators listed below
Calls and parentheses	<code>f(arg1, arg2, ...)</code> , <code>(e)</code>

A call argument is an expression, `&x`, or `&mut x`. References are only allowed in parameter types and these argument forms. A call target and a dereference operand must be identifiers: `(f)(x)`, `*(e)`, and `**p` are not supported. In this grammar, `*p[i]` means `(*p)[i]`. Function parameters, call arguments, and array elements may form empty lists, but trailing commas are not allowed.

Statements and blocks

The following forms are supported. `T` denotes a value type, and `{ ... }` denotes a block containing zero or more statements.

```
let x = e;           let x: T = e;
let mut x = e;       let mut x: T = e;
e;                   lhs = rhs;
if e { ... }         if e { ... } else { ... }
while e { ... }
println!("{}", e);
return;              return e;
```

Both sides of an assignment are expressions; whether the left side is assignable is checked later. Declarations require an initializer. Semicolons are required exactly as shown. There are no standalone block statements, empty `;` statements, implicit tail returns, or `else if` syntax. Use an `if` inside an `else` block when nesting conditions.

Precedence and associativity

Implement the following order, from lowest to highest. Parentheses override this order.

Operators	Associativity / form
<code> </code>	Left-associative
<code>&&</code>	Left-associative
<code>< > <= >= == !=</code>	One nonassociative level; reject unparenthesized chains
<code>+ -</code>	Left-associative
<code>* / %</code>	Left-associative
Unary <code>-</code> and <code>!</code>	Prefix operators; nesting is allowed
Array indexing <code>a[i]</code>	Highest precedence

For example, `1 + 2 * 3` means `1 + (2 * 3)`, `8 - 3 - 1` means `(8 - 3) - 1`, and `-a[i]` means `-(a[i])`. Reject `a < b < c` and `a < b == c` unless parentheses explicitly group a comparison.

2. Required Functions

Implement `lib/lexer.mll` so that `ocamllex` generates the following interface in `Lexer`:

```
val token : Lexing.lexbuf -> Parser.token
exception LexError of { msg : string; pos : Lexing.position }
```

Each call returns the next token, skipping whitespace and comments. Keep token-start positions correct after skipped input. With the supplied driver, line numbers and displayed byte columns are one-based; tabs count as one byte, not as expanded display columns.

Complete `lib/parser.mly` so that `Menhir` generates this entry point in `Parser`:

```
val program :
  (Lexing.lexbuf -> token) -> Lexing.lexbuf -> Ast.prog
```

The provided driver calls `Parser.program Lexer.token lexbuf`. The parser must consume the entire program, including EOF, and return an `Ast.prog`. Syntactically invalid input must be rejected.

3. Correctness

- For a valid input, return the token sequence and AST described by this specification. Preserve the order of functions, parameters, arguments, statements, and array elements.
- Use the supplied AST constructors. Parentheses group an expression without adding an AST node. Omitted local type annotations and absent return expressions are represented by `None`; present ones use `Some`. An omitted function return annotation becomes `T_unit`.
- Accept every syntactically valid program even if it contains type errors. For example, `let x: bool = 1;`, `if 1 {}`, and `1 = 2;` are valid statement syntax inside a function. Name resolution, duplicate names, type compatibility, and mutability checks belong to later assignments. A parser must not require a `main` function.
- Reject malformed syntax, such as missing delimiters or semicolons, trailing commas, and unparenthesized comparison chains. Adding whitespace or comments between tokens must preserve the AST when tokenization is unchanged; do not split a multi-character token such as `println!` or `>=`.

4. Implementation Requirements

- Only `lib/lexer.mll` and `lib/parser.mly` will be graded.
- Keep the supplied token names, payload types, AST definitions, exception, and entry points compatible with the driver.
- Add the missing lexer rules, parser alternatives, semantic actions, and precedence declarations. Keep the supplied examples and outer grammar structure. Helpers may be added inside the two implementation files.
- The project must build with the supplied Dune configuration. Use `--lex` and `--parse` to test this assignment; the typechecker, IR generator, and optimization pass may remain unfinished. You may add your own `.rs` test inputs.

Menhir notation used in the skeleton

Notation	Meaning
<code>option(X)</code>	Zero or one <code>X</code> ; returns <code>None</code> or <code>Some value</code>
<code>preceded(A, X)</code>	Reads <code>A</code> followed by <code>X</code> ; returns the value of <code>X</code>
<code>list(X)</code>	Zero or more <code>X</code> values, in source order
<code>separated_list(COMMA, X)</code>	Zero or more comma-separated <code>X</code> values, without a trailing comma

Thus, `option(preceded(COLON, typ))` reads an optional type annotation. `%left`, `%right`, and `%nonassoc` declare associativity and precedence; later declarations have higher precedence. `%prec UNOP` assigns a production the precedence declared for `UNOP`. Declare this marker before using it; the lexer never returns a `UNOP` token. Use the `LBRACKET` precedence to resolve indexing against unary and binary operators.

These are Menhir features, not RustY syntax. See the Menhir Reference Manual, Sections 4.1.4, 4.2.1, and 5.4. When writing binary-expression rules, use care if factoring operators into another nonterminal: precedence is not automatically inherited through an ordinary nonterminal.

5. Examples

Tokenization

```
let mut total: i32 = -12;
```

Ignoring the locations printed by the driver, the token sequence is:

```
LET MUT IDENT("total") COLON I32 EQ MINUS INT(12) SEMICOLON EOF
```

Parsing

Save the following program as `sample.rs` in the project root, the directory containing `dune-project`:

```
fn twice(x: i32) -> i32 {
    return x * 2;
}

fn main() -> i32 {
    let y = twice(1 + 2 * 3);
    println!("{}", y);
    return 0;
}
```

The argument expression `1 + 2 * 3` must produce the following expression subtree:

```
E_Binop (Add,
  E_Lit (Num 1),
  E_Binop (Mul, E_Lit (Num 2), E_Lit (Num 3)))
```

Rejected inputs

Input	Expected result
@	Lexer error: unsupported character
fn f() { let x = 1 }	Parser error: missing semicolon
fn f() { let a = [1, 2,]; }	Parser error: trailing comma
fn f() { 1 < 2 < 3; }	Parser error: comparison chain

6. Tests

From the project root, using the course's configured opam environment:

```
dune build
dune exec rustyc -- --lex sample.rs
dune exec rustyc -- --parse sample.rs

# Save successful output to files.
dune exec rustyc -- --lex sample.rs -o tokens.txt
dune exec rustyc -- --parse sample.rs -o parsed.txt
```

`--lex` prints token names, payloads, and starting positions. `--parse` prints the AST back in source-like form using the supplied printer; it does not display OCaml constructor names. Both write to standard output unless `-o` is given. Errors go to standard error and produce a nonzero exit status.

Add tests for keyword prefixes, adjacent multi-character operators, comments with and without a final newline, nested arrays and indexing, reference parameters and arguments, empty lists, precedence, associativity, and malformed inputs. Also test syntactically valid programs with type errors. Successful parsing alone does not establish that operator grouping is correct; inspect the printed grouping or the AST.

`dune runtest` runs the tests registered in the project's Dune files. It does not automatically discover every `.rs` file, and a passing run does not establish that the lexer and parser are complete.